

Computer Science I

Week 2

Tutorial Session

Coding Exercise 1 - Reverse an Array

Write a function:

```
void reverse_array(int a[], int n);
```

The function must reverse the first n elements of a in place. You may not create a second array.

When an array is passed to a function, write the array parameter using square brackets, as in `int a[]`. The function works with the same array as the caller. Therefore, if the function changes an element such as `a[0]`, that element is also changed in the original array. The length of the array is not passed automatically, so pass n as a separate argument.

Examples:

```
[4, 7, 2, 9, 1]  -> [1, 9, 2, 7, 4]
[3, 8, 5, 6]     -> [6, 5, 8, 3]
[10]             -> [10]
```

Write a complete program that defines the function, calls it on an example array, and prints the array before and after the call.

Coding Exercise 1a - Reverse an Array Recursively

Solve the same problem using recursion. Write a function:

```
void reverse_recursive(int a[], int start, int end);
```

The function reverses the portion of the array between `start` and `end`, inclusive. It must not contain a `for` or `while` loop. For an array `a` of length n , the initial call is:

```
reverse_recursive(a, 0, n - 1);
```

Hint: think about the two elements at the ends of the interval. After swapping them, which smaller interval still needs to be reversed? Use the following structure:

```
void reverse_recursive(int a[], int start, int end)
{
    /* base case */

    /* swap two elements */

    /* recursive call on a smaller interval */
}
```

Determine when the recursion stops, which two elements are swapped, and how `start` and `end` change in the recursive call.

Coding Exercise 2 - Longest Increasing Run

Write a function:

```
int longest_increasing_run(int a[], int n);
```

Assume that $n > 0$. A run is a sequence of consecutive array elements. The function must return the length of the longest run in which every element is strictly greater than the previous one.

Examples:

```
[2, 4, 7, 3, 5, 6, 8, 1] -> 4  
[5, 4, 3, 2]             -> 1  
[1, 2, 2, 3]             -> 2
```

In the first example, the longest increasing run is 3, 5, 6, 8.

Write a complete program that tests the function on the three examples above.

Coding Exercise 3 - Repeated Digit Sum

Write a function:

```
int digital_root(int n);
```

Assume that $n > 0$. Replace n with the sum of its decimal digits. Repeat this process until only one digit remains, then return that digit.

For example:

```
583 -> 5 + 8 + 3 = 16  
16  -> 1 + 6 = 7
```

Therefore:

```
digital_root(583) returns 7  
digital_root(9999) returns 9  
digital_root(7)   returns 7
```

Use loops, integer division, and the remainder operator `%`. Do not use strings or arrays. Write a complete program that tests the three examples above.

Coding Exercise 4 - Greatest Common Divisor

The greatest common divisor (GCD) of two positive integers is the largest positive integer that divides both numbers exactly. Euclid's algorithm uses the following idea: dividing a by b gives a remainder. Replacing a and b with b and this remainder does not change their GCD. Repeat this process until the remainder is 0. At that point, the other number is the GCD.

For example, starting from 48 and 18:

```
gcd(48, 18)
-> gcd(18, 12)
-> gcd(12, 6)
-> gcd(6, 0)
-> 6
```

Write a recursive function:

```
int gcd(int a, int b);
```

Assume that $a > 0$ and $b \geq 0$. The function must use recursion and must not contain a loop.

Use the following recursive rule:

```
gcd(a, b) = gcd(b, a % b)    when b != 0
gcd(a, 0) = a
```

Test at least these cases:

```
gcd(48, 18) returns 6
gcd(21, 14) returns 7
gcd(17, 5)  returns 1
```

Coding Exercise 5 - Recursive Binary Search

Binary search looks for a value in a sorted array by examining the middle element of the current search interval. If the middle element is not the target, the ordering of the array tells us which half can still contain the target; the other half can be discarded.

Write a recursive function:

```
int binary_search(const int a[], int left, int right, int target);
```

The array is sorted in increasing order. The function must return the index of target if it occurs between indices left and right, inclusive. If the target is not present, return -1. The function must use recursion and must not contain a loop.

For the array:

```
int a[] = {2, 5, 8, 12, 17, 21, 30};
```

the following calls should produce:

```
binary_search(a, 0, 6, 17) returns 4  
binary_search(a, 0, 6, 2)  returns 0  
binary_search(a, 0, 6, 15) returns -1
```

Hint: first identify the base case for an empty search interval. Otherwise, inspect the middle element and recursively search only the half that can still contain the target.

```
/* base case */  
  
/* compute and inspect the middle element */  
  
/* recursively search the appropriate half */
```

Write a complete program that tests the three calls above.